

Robustesse et fraîcheur du corpus

Fiche pédagogique – Corrections post-démo

Système RAG pour hcp.ma

Notes personnelles, à des fins d'apprentissage

Cinq problèmes trouvés après un test en conditions réelles avec l'encadrante, cinq corrections expliquées en détail

Table des matières

1	Contexte	2
2	Problème 1 : un plantage silencieux sur simple panne réseau	2
2.1	Ce qui a été observé	2
2.2	Pourquoi	2
2.3	Le correctif	3
3	Problème 2 : un corpus figé depuis des semaines	3
3.1	Ce qui a été observé	3
3.2	Pourquoi	3
3.3	Le correctif	4
4	Problème 3 : relancer l’indexation la pollueait	4
4.1	Ce qui a été observé	4
4.2	Pourquoi	4
4.3	Le correctif	5
5	Problème 4 : le cache peut resservir une réponse périmée	5
5.1	Le dilemme	5
5.2	Options écartées, et pourquoi	5
5.3	La décision retenue	5
6	Problème 5 : un historique qui ne s’effaçait jamais	6
6.1	Ce qui a été observé	6
6.2	Le correctif	6
7	Une décision à part : garder le log de diagnostic	7
8	Les tests : ce qui a été validé	7
9	Synthèse – le fil conducteur	7
10	Ce qui reste ouvert	8

1 Contexte

Un test en conditions réelles avec l'encadrante a mal tourné : des réponses hors sujet, puis la même réponse resservie à des questions différentes, dans une session comme dans une autre. Rien de tout ça n'était visible dans les 194 tests automatisés qui passaient au vert – encore une fois la même leçon déjà tirée plusieurs fois sur ce projet : les tests unitaires valident la *logique*, seule l'utilisation réelle révèle ce à quoi la conception n'avait pas pensé.

La démarche de diagnostic, avant toute correction : ajouter un log temporaire ([`DEBUG reformulation`]) pour voir exactement ce que le système fait en interne, reproduire le scénario, lire la vraie base de données et le vrai contenu de Redis plutôt que de deviner. Plusieurs hypothèses initiales (la reformulation qui dérape, le cache qui mélange les sessions) se sont révélées fausses ou incomplètes en testant – les vrais coupables étaient ailleurs, et il a fallu les chercher un par un.

Cinq problèmes distincts ont fini par être identifiés et corrigés. Cette fiche les détaille dans l'ordre où ils ont été trouvés.

2 Problème 1 : un plantage silencieux sur simple panne réseau

2.1 Ce qui a été observé

En testant plus de questions, une erreur est apparue dans le terminal : une `ConnectionError` Python (module `requests`), suivie de l'arrêt complet de l'interface Streamlit. Aucune réponse affichée, toute la session perdue.

2.2 Pourquoi

Le module `Generateur` appelle Mistral pour rédiger la réponse sur le chemin NOTION (et pour la partie explicative du chemin MIXTE). Partout ailleurs dans le projet, un appel à un LLM externe est entouré d'un `try/except` qui retombe proprement sur un comportement par défaut si l'appel échoue (voir le Routeur et le Reformulateur). Ici, non : rien ne protégeait cet appel précis. La moindre coupure réseau, le moindre hoquet de l'API Mistral, faisait remonter l'exception jusqu'en haut de la pile et tuait tout le processus Streamlit – pour tout le monde, pas seulement pour la question en cours.

```
1  AVANT :
2  Question NOTION posee
3      |
4      v
5  Generateur.generer_reponse()
6      |
7      v
8  fonction_generation(question, contexte) -- appel reseau vers Mistral
9      |
10     X  ConnectionError (reseau coupe)
11     |
12     v
13  PLANTE tout le processus Streamlit
14  (aucune reponse affichee, session perdue pour tous)
15
16  APRES :
17  Question NOTION posee
18      |
19      v
20  Generateur.generer_reponse()
21      |
22      v
23  try: fonction_generation(question, contexte)
24      |
```

```

25  +-----+-----+
26  v             v
27  OK           ConnectionError
28  |             |
29  v             v
30  Reponse      Message d'erreur explicite (NOTION)
31  normale      ou chiffre seul conserve (MIXTE)
32  |             |
33  +-----+-----+
34              v
35  Interface reste utilisable

```

2.3 Le correctif

Un `try/except` autour de l'appel, avec une dégradation différente selon le chemin : sur NOTION, un message d'erreur explicite plutôt qu'un plantage. Sur MIXTE, mieux encore : le chiffre officiel ne dépend d'aucun LLM (produit par un gabarit déterministe), donc une panne de l'explication ne doit jamais priver l'utilisateur du chiffre déjà calculé.

```

1  # src/generateur.py -- chemin NOTION
2  texte_contexte = "\n\n".join(chunk.texte for chunk in chunks)
3  try:
4      texte = self._fonction_generation(question, texte_contexte)
5  except Exception:
6      return Reponse(
7          texte=(
8              "Une erreur technique (reseau ou service de generation "
9              "indisponible) a empeche de repondre a cette question. "
10             "Reessayez dans quelques instants."
11         ),
12         source_url="", source_titre="", source_date=None,
13     )
14
15 # src/generateur.py -- chemin MIXTE
16 try:
17     explication = self._fonction_generation(question, texte_contexte)
18 except Exception:
19     # Le chiffre officiel reste fiable, produit sans aucun LLM --
20     # une panne de l'explication ne doit pas priver l'utilisateur du chiffre.
21     return Reponse(texte=phrase_chiffree, source_url=url_chiffre,
22                    source_titre=titre_chiffre, source_date=date_chiffre)

```

Deux nouveaux tests reproduisent une panne réseau simulée (`ConnectionError` levée volontairement par une fausse fonction de génération) sur les deux chemins : **Validé en réel**.

3 Problème 2 : un corpus figé depuis des semaines

3.1 Ce qui a été observé

L'encadrante a interrogé le système sur une publication récente : rien trouvé, ou une réponse construite à partir d'un contenu périmé. En vérifiant directement la base de données, le document le plus récemment publié qui soit indexé datait de plusieurs semaines – rien de plus récent n'existait dans le système.

3.2 Pourquoi

L'automatisation de la mise à jour était prévue dès le début du projet (voir l'ADR sur l'API BDS : *"un script planifié, ex. une fois par jour"*), mais cette étape n'avait jamais été réellement câblée à un vrai planificateur – ni tâche Windows, ni cron. Les scripts existaient (`indexer_documents.py`,

`preremplir_indicateurs_bds.py`), mais rien ne les déclenchait tout seul : ils devaient être relancés à la main, et ça n'avait pas été fait depuis longtemps.

3.3 Le correctif

Un script d'orchestration Python enchaîne les deux étapes (voir le code plus bas), appelé chaque nuit par une véritable tâche planifiée Windows, via un petit script PowerShell intermédiaire.

```
1 Planificateur de taches Windows (chaque nuit)
2   |
3   v
4 rafraichir_corpus.ps1
5   |
6   v
7 python -m scripts.rafraichir_corpus
8   |
9   +--> indexer_documents.main()
10      |      parcourt la page 1 de chaque categorie hcp.ma
11      |
12      +--> preremplir_indicateurs_bds.main()
13          |      met a jour les 18 indicateurs BDS cures
14          |
15      +--> (voir Probleme 4 -- decide s'il faut vider le cache)
```

```
1 schtasks /create /tn "RAG_HCP_Rafraichissement_Nocturne" ^
2   /tr "powershell.exe -ExecutionPolicy Bypass -File rafraichir_corpus.ps1" ^
3   /sc daily /st 03:00
```

Limite assumée : cette tâche ne s'exécute que si le PC est allumé à l'heure prévue – un palliatif pour la phase de développement sur machine personnelle. Une fois déployé sur un vrai serveur (24/7 par définition), un cron classique remplace directement le Planificateur Windows, sans changer une ligne du code Python.

Testé en conditions réelles : le corpus est passé de 96 à 99 documents (7499 chunks) dès le premier rafraîchissement – **Validé en réel**.

4 Problème 3 : relancer l'indexation la polluit

4.1 Ce qui a été observé

En préparant le correctif précédent, un examen du code d'`indexer_documents.py` a révélé un second bug, jamais rencontré jusque-là parce que ce script n'avait jamais été relancé plusieurs fois sur le même corpus.

4.2 Pourquoi

La page 1 de chaque listing hcp.ma contient toujours un mélange de publications déjà connues et de vraies nouveautés (le site trie du plus récent au plus ancien, mais une page entière dépasse presque toujours ce qui a changé depuis la dernière fois). La contrainte `document.url` UNIQUE empêchait bien un doublon dans la table `document`, mais rien n'empêchait de refaire extraction, découpage et calcul des embeddings pour un document *déjà indexé*, et de réinjecter des chunks en double dans la base et l'index – coûteux, et ça polluit le corpus à chaque relance.

1 AVANT (a chaque relance) :	APRES :
2 Page 1 rencontrée	Page 1 rencontrée
3	
4 v	v
5 Déjà connue ? oui	Déjà connue ? oui
6	

<pre> 7 V 8 Reextraction + reembedding 9 + chunks dupliques en base </pre>	<pre> 7 V 8 ARRET immediat, rien de plus 9 (statut "deja_indexe") </pre>
---	---

4.3 Le correctif

Une vérification précoce, avant tout travail coûteux : si l'URL est déjà en base, on s'arrête net.

```

1  # scripts/indexer_documents.py
2  deja_connu = conn.execute(
3      "SELECT id_document FROM document WHERE url = ?", (document.url,)
4  ).fetchone()
5  if deja_connu is not None:
6      return {"statut": "deja_indexe", "chunks": 0, "tableaux": 0,
7              "id_document": deja_connu[0]}
8
9  texte, tableaux = extracteur.extraire(document)  # seulement si vraiment nouveau

```

Un test existant vérifiait déjà qu'une même URL ne créait pas deux lignes `document` – mais aucun test ne vérifiait le nombre de `chunks`, ce qui avait laissé ce bug invisible. Un nouveau test comble ce trou précis : **Validé en réel**.

5 Problème 4 : le cache peut resservir une réponse périmée

5.1 Le dilemme

Le cache des réponses NOTION (Redis) est volontairement partagé entre toutes les sessions : si une personne demande "pourquoi le chômage a-t-il augmenté ?" aujourd'hui, une autre personne qui pose la même question demain doit pouvoir bénéficier d'une réponse déjà calculée, sans rappeler le LLM inutilement – c'est le but assumé du cache, pas un oubli à corriger.

Mais un TTL de 24h glissant depuis l'écriture de l'entrée n'est pas aligné sur le moment réel où le corpus change : le rafraîchissement nocturne tourne à heure fixe (3h), alors qu'une entrée mise en cache la veille à 14h reste valide jusqu'à 14h le lendemain – jusqu'à 11h de décalage où le cache peut resservir une réponse plus vieille que du contenu déjà mis à jour.

5.2 Options écartées, et pourquoi

Cloisonner le cache par session (une clé Redis par `id_session`) réglait le problème de fraîcheur, mais au prix du bénéfice de partage entre sessions – écarté, ce bénéfice étant justement ce qui était recherché dès le départ.

Une invalidation ciblée (ne supprimer que les entrées concernées par un nouveau document) demanderait de savoir, pour chaque réponse en cache, quels documents ont servi à la produire – puis de comparer sémantiquement chaque nouveau document à chaque question en cache, en utilisant la même machinerie que la recherche (embeddings, découpage, fusion RRF). Cette machinerie existe déjà dans le projet, mais elle ne tourne qu'au moment où un utilisateur pose une vraie question – la déclencher pour comparer chaque nouveau document à tout le cache serait coûteux, et pas fiable à 100 % de toute façon. Écarté.

5.3 La décision retenue

Garder le cache global, mais le vider entièrement – sans distinction de sujet – dès qu'au moins une vraie nouveauté est trouvée pendant le rafraîchissement nocturne. Simple, fiable à 100 %, et n'affecte pas les nuits où rien n'a changé.

```

1  Rafraichissement nocturne termine
2  |

```

```

3         v
4 Au moins 1 nouveau document trouve ?
5         |
6     +-----+-----+
7     v             v
8     oui           non
9     |             |
10    v             v
11 Cache NOTION     Cache NOTION
12 vide entier      conserve tel quel
13 (toutes cles)    (rien ne change)

```

```

1 # scripts/vider_cache_notion.py
2 def vider(client_redis) -> int:
3     if client_redis is None:
4         return 0
5     cles = list(client_redis.scan_iter(PREFIXE_CACHE + "*"))
6     if cles:
7         client_redis.delete(*cles)
8     return len(cles)
9
10 # scripts/rafraichir_corpus.py -- decision
11 nouveaux_documents = indexer_documents.main(chemin_db=chemin_db)
12 ...
13 if nouveaux_documents > 0:
14     client = _connecter_redis()
15     vider(client)
16 else:
17     print("Aucune nouveaute -> cache NOTION conserve.")

```

Limite assumée, discutée explicitement : hcp.ma ne semble jamais réviser un document existant sous la même URL, seulement publier de nouveaux documents avec des chiffres à jour. Un nouveau document peut donc rendre une réponse en cache obsolète sur un sujet qu'il ne partage avec *aucun* document déjà cité dans le cache – un suivi "quel document a servi à quelle réponse" ne suffirait pas à lui seul à cibler quoi invalider. Vider tout dès qu'il y a une nouveauté reste la seule garantie fiable sans construire la comparaison sémantique complète évoquée plus haut.

4 nouveaux tests (suppression effective, historique jamais touché, cache vide ou absent géré proprement) : **Validé en réel.**

6 Problème 5 : un historique qui ne s'effaçait jamais

6.1 Ce qui a été observé

En inspectant directement Redis (`redis-cli`, ou l'équivalent en Python), les clés d'historique de conversation affichaient toutes TTL: -1 – aucune expiration. Elles s'accumulaient indéfiniment, contrairement au cache qui, lui, avait toujours eu 24h.

6.2 Le correctif

Un TTL glissant de 24h, repoussé à chaque nouveau message de la session plutôt que fixé une seule fois à la création – une conversation active ne doit jamais expirer en plein milieu, seule une session vraiment abandonnée finit par disparaître.

```

1 Session active :
2     message 1 -> rpush + expire(24h)           TTL = 24h
3     message 2 -> rpush + expire(24h)           TTL reparti a 24h (pas de coupure)
4     ...
5
6 Session abandonnee (onglet ferme sans rien faire d'autre) :

```

```

7 aucun nouveau message -> TTL continue de descendre -> 0 -> cle supprimee

1 # src/cache_redis.py
2 def ajouter(self, id_session: str, question: str, reponse: Reponse) -> None:
3     if self._client is None:
4         return
5     entree = json.dumps({"question": question, "reponse": asdict(reponse)})
6     cle = PREFIXE_HISTORIQUE + id_session
7     self._client.rpush(cle, entree)
8     self._client.expire(cle, TTL_HISTORIQUE) # 24h, repousse a chaque ajout

```

Une alternative envisagée – supprimer l'historique à la fermeture exacte de l'onglet – a été écartée : Streamlit n'offre aucun signal fiable côté serveur pour détecter ce moment précis (pas de vraie notification de déconnexion exploitable simplement). Un bouton "Nouvelle conversation" explicite reste possible plus tard pour le cas où l'utilisateur veut clore volontairement – pas construit pour l'instant, le TTL glissant étant jugé suffisant.

2 nouveaux tests (TTL posé à la création, TTL repoussé à chaque nouvel ajout) : **Validé en réel.**

7 Une décision à part : garder le log de diagnostic

Le log [DEBUG reformulation] ajouté pour cette enquête (affiche dans le terminal, à chaque reformulation, le texte avant et après) a failli être retiré une fois le diagnostic terminé – puis conservé volontairement : il répond exactement à un besoin de visibilité déjà demandé pour cette phase de développement, sans qu'il ait été évident au départ que c'était techniquement possible aussi simplement.

```

1 # scripts/poser_question.py
2 question_effective = reformuler_si_necessaire(question, entrees_recentes,
3     fonction_reformulation)
4 if question_effective != question:
5     print(f"[DEBUG reformulation] '{question}' -> '{question_effective}'")
6 type_question = routeur.classifier(question_effective)

```

8 Les tests : ce qui a été validé

Correctif	Tests	Ce qui est prouvé
Dégradation propre du <code>Generateur</code>	2	Panne réseau simulée sur NOTION (message d'erreur, pas de plantage) et sur MIXTE (chiffre conservé).
Anti-duplication <code>indexer_documents</code>	1	Une même URL relancée deux fois ne crée plus de chunks en double.
<code>vider_cache_notion</code>	4	Suppression effective du cache, historique jamais touché, cas sans client géré proprement.
TTL glissant sur l'historique	2	TTL de 24h posé à la création, repoussé à chaque nouveau message.

Suite complète du projet : **194 tests avant ces corrections** → **196 tests désormais**, tous verts. Aucune régression sur l'existant à aucune étape.

9 Synthèse – le fil conducteur

Un test réel avec un tiers vaut plus que des semaines de tests solo. Les cinq problèmes de cette fiche existaient depuis un moment, invisibles en testant seul avec les mêmes questions habituelles – il a fallu quelqu'un d'extérieur, avec des questions imprévues, pour les faire remonter d'un coup.

Diagnostiquer avant de corriger, toujours. Plusieurs pistes initiales (reformulation qui dérape, cache qui mélange les sessions) se sont révélées fausses ou incomplètes une fois vérifiées avec un vrai log et une vraie inspection de Redis. Corriger à l’aveugle sur une hypothèse non vérifiée aurait pu masquer le vrai problème au lieu de le résoudre.

La dégradation propre doit être systématique, pas au cas par cas. Le Routeur et le Reformulateur avaient déjà ce réflexe (un `try/except` autour de chaque appel externe) – le **Generateur** était le seul endroit oublié, et ça a suffi à faire planter toute l’interface. Une règle de conception n’a de valeur que si elle est appliquée partout, sans exception oubliée.

Simple et fiable bat sophistiqué et fragile. Vider tout le cache dès qu’une nouveauté apparaît est moins élégant qu’une invalidation ciblée par sujet – mais cette dernière demanderait une comparaison sémantique coûteuse et jamais garantie à 100 %. À l’échelle de ce projet, la solution simple est aussi la plus sûre.

10 Ce qui reste ouvert

- **Découverte de nouveautés limitée à la page 1** de chaque catégorie – cohérent avec l’objectif "fraîcheur quotidienne", mais à surveiller si hcp.ma publie un jour plusieurs pages de nouveautés en une seule nuit (rafraîchissement manqué, par exemple).
- **Couverture des pages de listing** pas encore vérifiée manuellement contre le vrai site – à confirmer qu’aucune catégorie ou sous-section active n’est absente de `data/listing_urls.py`.
- **Bouton "Nouvelle conversation"** évoqué mais pas construit – le TTL glissant de 24h a été jugé suffisant pour l’instant.
- **Jeu de test de fiabilité** toujours à construire (30 à 50 questions-réponses, mesure du taux d’exactitude) – prochaine étape, une fois ces corrections considérées stables.